

Document Summary

This note covers robot motor control and stability analysis, including regulation control, trajectory tracking, Lyapunov stability theory, PID control, and Linear Quadratic Regulator (LQR) design for robotic systems.

Robot Motor Control and Stability Analysis

Lecture 3a: Stability and Regulation

Robot Control

Robotics: The intelligent connection of perception to action

- From "The intelligent connection of perception to action", J. M. Bradley, MIT AI Lab, 1986

Motion Control Objectives

Motion control defines how an autonomous system moves. Control objectives classify into three main classes:

1.
Regulation: Reach and maintain a desired fixed configuration (joint or task space). Transient behavior not guaranteed.
 - start → goal
2.
Trajectory Tracking: Follow a desired time-varying reference trajectory (task or joint space).
 - start → Reference trajectory → goal
3.
Contact Motion: Physically interact with environment, exchanging forces.

Joint Space Control

- Task specification (end-effector motion/forces) in operational space, control actions (joint actuator forces) in joint space.
- Objectives in task space converted to joint space using `inverse kinematics`.

Task Space Control

Task space control based on operational space dynamics:

$$\dot{x} = J(q)\dot{q}$$

$$\mathbf{f}_c = \mathbf{\Lambda}_q \ddot{\mathbf{x}} + \mathbf{\Gamma}_q \dot{\mathbf{x}} + \boldsymbol{\mu}(\mathbf{q})$$

$$\boldsymbol{\tau} = \mathbf{B}_q \ddot{\mathbf{q}} + \mathbf{C}_q(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$$

- Joint space dynamics
- Velocity mapping

Equilibrium States

Non-linear dynamical system:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{h}(\mathbf{x})\mathbf{u}$$

Equilibrium: System stays in state once reached (without perturbations or with suitable input).

- **Unforced equilibrium:** $\mathbf{u} = \mathbf{0} : \mathbf{f}_x^e = \mathbf{0}$
- **Forced equilibrium:** $\mathbf{u} = \mathbf{u}(\mathbf{x}) : \mathbf{f}_x^e + \mathbf{h}_x^e \mathbf{u}(\mathbf{x}^e) = \mathbf{0}$

Stability

Stability of $\mathbf{x}_e$

- Nonlinear system: $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$
- Equilibrium: $\mathbf{x}^* (\mathbf{f}(\mathbf{x}^*) = \mathbf{0})$

$$\forall \epsilon > 0 : \exists \delta > 0 : \|\mathbf{x}(t_0) - \mathbf{x}^*\| < \delta \Rightarrow \|\mathbf{x}(t) - \mathbf{x}^*\| < \epsilon, \forall t \geq t_0$$

Asymptotic Stability

Asymptotic stability of $\mathbf{x}_e$

- Nonlinear system: $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$
- Equilibrium: $\mathbf{x}^* (\mathbf{f}(\mathbf{x}^*) = \mathbf{0})$

$$\exists \delta > 0 : \|\mathbf{x}(t_0) - \mathbf{x}^*\| < \delta \Rightarrow \|\mathbf{x}(t) - \mathbf{x}^*\| \rightarrow 0, \text{ for } t \rightarrow \infty$$

$\mathbf{x}^*$ stable +

Exponential Stability

Exponential stability of $\mathbf{x}_e$

$$\exists \delta, c, \lambda > 0 : \|\mathbf{x}(t_0) - \mathbf{x}^*\| < \delta \Rightarrow \|\mathbf{x}(t) - \mathbf{x}^*\| \leq ce^{-\lambda(t-t_0)} \|\mathbf{x}(t_0) - \mathbf{x}^*\|$$

- Allows convergence time estimation: for $c = 1, t - t_0 = \ln(2)/\lambda$ reduces initial error by 50%
- Typically local property; domain of attraction hard to estimate.

How to Check Stability?

Assessing nonlinear system stability is non-trivial. Cannot enumerate all trajectories.

Example:

$$\dot{x}_1 = 1 - x_1^2$$

$$\dot{x}_2 = x_1 - x_2^2$$

System: $\dot{x} = f(x)$

Equilibria:

$$x_1^* = (1, 1)$$

$$x_2^* = (1, -1)$$

Lyapunov

Lyapunov candidate

$V(x) : \mathbb{R}^n \rightarrow \mathbb{R}$ s.t. $V(x^*) = 0, V(x) > 0, \forall x \neq x^*$

- V positive definite. Typical choice: quadratic $(x - x^*)^T P (x - x^*)$
- V may be local: $\forall x : \|x - x^*\| < \delta$

Lyapunov functions provide stability conditions without enumerating trajectories!

Lyapunov Conditions

System $\dot{x} = f(x)$, Lyapunov candidate V for x^* :

- **Stability sufficient condition:** $\exists V : \dot{V}(x) \leq 0$ along trajectories
- **Asymptotic stability sufficient condition:** $\exists V : \dot{V}(x) < 0$ along trajectories
- **Instability sufficient condition:** $\exists V : \dot{V}(x) > 0$ along trajectories

LaSalle Theorem

LaSalle Theorem

If $\exists V$ candidate : $\dot{V}(x) \leq 0$ along trajectories

Then system trajectories asymptotically converge to largest invariant set $\mathcal{M} \subseteq \mathcal{S} = \{x \in \mathbb{R}^n : \dot{V}(x) = 0\}$

- $\mathcal{M}$ invariant if $x(t_0) \in \mathcal{M} \Rightarrow x(t) \in \mathcal{M}, \forall t \geq t_0$
- $\mathcal{M} \equiv \{x^*\}$ implies asymptotic stability

Stability of Linear Systems

$$\dot{x} = Ax$$

$\mathbf{x}^* = \mathbf{0}$ always equilibrium.

1. Asymptotic stability
2. Global asymptotic stability
3. Exponential stability
4. $\sigma(\mathbf{A}) \subset \mathbb{C}^-$ (all eigenvalues have negative real part)
5. $\forall \mathbf{Q} \succ 0, \exists \mathbf{P} \succ 0 : \mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} = -\mathbf{Q}$ and $(\mathbf{x} - \mathbf{x}^*)^T \mathbf{P} (\mathbf{x} - \mathbf{x}^*)$ is Lyapunov candidate

ALL EQUIVALENT!

If $\mathbf{x}^* = \mathbf{0}$ asymptotically stable, it is unique equilibrium.

Lyapunov equation:

$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} = -\mathbf{Q}$$

Linear Approximation

Let $\Delta \mathbf{x} = \mathbf{x} - \mathbf{x}^*$ and $\dot{\Delta \mathbf{x}} = \mathbf{A} \Delta \mathbf{x}$ be linear approximation of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ around $\mathbf{x}^*$.

- $\mathbf{A}$ asymptotically stable ($\sigma(\mathbf{A}) \subset \mathbb{C}^-$)
- Original nonlinear system exponentially stable at origin

Note: Local result only.

Regulation Control

Control

Robot $\mathbf{B}_q \ddot{\mathbf{q}} + \mathbf{C}_q(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}$

Goal: Asymptotic stabilization of closed-loop equilibrium: $\mathbf{q} = \mathbf{q}_d$ with $\dot{\mathbf{q}} = \mathbf{0}$.

$\mathbf{q}_d$ possibly from kinematics: $\mathbf{q}_d = \mathbf{f}^{-1}(\mathbf{x}_d)$

PD control law:

$$\mathbf{u} = \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) - \mathbf{K}_d \dot{\mathbf{q}}$$

with $\mathbf{K}_p, \mathbf{K}_d > 0$ positive definite symmetric.

Example: PD for Serial Manipulator

Without gravity ($\mathbf{g}(\mathbf{q}) \equiv \mathbf{0}$), robot state $(\mathbf{q}_d, \mathbf{0})$ under PD joint control globally asymptotically stable.

Proof:

- $\mathbf{e} = \mathbf{q}_d - \mathbf{q}$
- $V = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{B}_q \dot{\mathbf{q}} + \frac{1}{2} \mathbf{e}^T \mathbf{K}_p \mathbf{e} \geq 0$

$$V = 0 \iff e = \dot{e} = 0$$

$$\begin{aligned}\dot{V} &= \dot{q}^T B_q \ddot{q} + \frac{1}{2} \dot{q}^T \dot{B}_q \dot{q} - e^T K_p \dot{q} = \dot{q}^T u - \dot{q}^T C_q \dot{q} + \frac{1}{2} \dot{q}^T \dot{B}_q \dot{q} - e^T K_p \dot{q} \\ &= \dot{q}^T K_p e - \dot{q}^T K_d \dot{q} - \dot{q}^T C_q \dot{q} + \frac{1}{2} \dot{q}^T \dot{B}_q \dot{q} - e^T K_p \dot{q} \\ &= -\dot{q}^T K_d \dot{q} \leq 0\end{aligned}$$

Proves stability only (equals zero if C built using Christoffel symbols).

Example: PD for Serial Manipulator (Continued)

$$B_q \ddot{q} + C_q(q, \dot{q}) \dot{q} + g(q) = K_p e - K_d(\mathbf{0} - \dot{q})$$

Proven simple stability, but $\dot{V} = 0 \Leftrightarrow \dot{q} = \mathbf{0}$.

Thus $\dot{q} = \mathbf{0}, \ddot{q} = \mathbf{0} \Leftrightarrow e = \mathbf{0}$.

From LaSalle theorem, trajectories converge to largest invariant set $\mathcal{M}$ where $\dot{q} = \mathbf{0}$ ($\dot{q} = \ddot{q} = \mathbf{0}$).

$$\dot{q} = \mathbf{0}$$

$$\ddot{q} = B_q^{-1} K_p e$$

Only invariant state in $\dot{V} = 0$ is $q = q_d, \dot{q} = \mathbf{0}$.

Physical Interpretation

For diagonal positive definite K_p and K_d , values correspond to stiffness of "virtual" springs and viscosity of "virtual" dampers at joints.

Plot of the Lyapunov Function

Time evolution of Lyapunov candidate.

Inclusion of Gravity

- **With gravity**, modify PD controller adding gravity compensation:

$$u = K_p(q_d - q) - K_d \dot{q} + g(q)$$

- **However**, if gravity approximately compensated:

$$u = K_p(q_d - q) - K_d \dot{q} + \hat{g}(q)$$

with $\hat{g}(q) \neq g(q)$

then $q \rightarrow q^* \neq q_d, \dot{q} \rightarrow \mathbf{0}$ (steady-state position error).

- Equilibrium q^* generally not unique; $q^* \rightarrow q_d$ when $K_p \rightarrow \infty$.

PID Control

Compared to PD, PID adds **integral control** to eliminate constant steady-state error in step response (linear systems).

- **Example:** Use manipulator to recover error from absent/incomplete gravity compensation:

$$\mathbf{u}(t) = \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_i \int_0^t (\mathbf{q}_d - \mathbf{q}(\tau)) d\tau - \mathbf{K}_d \dot{\mathbf{q}}(t)$$

- **If** desired closed-loop equilibrium asymptotically stable under PID, integral term compensates gravity at steady state.

Comments on PID Control

- **Control gains** $\mathbf{K}_p$ and $\mathbf{K}_d$ affect transients and settling times.
- Hard to define optimal values, especially for whole workspace.
- "full" $\mathbf{K}_p$ and $\mathbf{K}_d$ matrices assign desired eigenvalues to linear approximation around $(\mathbf{q}_d, \mathbf{0})$.
- **Viscous friction** $-\mathbf{F}_v \dot{\mathbf{q}}$ acts like derivative term $-\mathbf{K}_d \dot{\mathbf{q}}$.

Comments on PID Control (Continued)

- **Response times** to reach desired state not easily predictable.
- **Integral term** needs time to "unload" from transient error history (anti-windup/saturation possible).

Linear Quadratic Regulator

Linear Quadratic Regulator (LQR)

- LQR is **optimal control** scheme for stabilizing time-invariant linear system to origin.

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$$

Dynamical system

$$J(\mathbf{X}, \mathbf{U}) = \sum_{k=0}^{N-1} [\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k] + \mathbf{x}_N^T \mathbf{Q}_f \mathbf{x}_N$$

Quadratic cost function over finite horizon

where $\mathbf{Q}, \mathbf{R}, \mathbf{Q}_f \succ 0$ symmetric.

- **State cost:** $\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k$
- **Final state cost:** $\mathbf{x}_N^T \mathbf{Q}_f \mathbf{x}_N$
- **Input cost:** $\mathbf{u}_k^T \mathbf{R} \mathbf{u}_k$

Positive definite: $\mathbf{x}^T \mathbf{Q} \mathbf{x} > 0 \forall \mathbf{x} \neq \mathbf{0}$.

Symmetric Matrices

- Symmetric $\mathbf{Q}$: $\mathbf{Q} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$ where $\mathbf{\Lambda}$ diagonal, $\mathbf{V}^T \mathbf{V} = \mathbf{I}$
- Quadratic form rotates via $\mathbf{V}$, rescales via $\mathbf{\Lambda}$:

$$\mathbf{x}^T \mathbf{Q} \mathbf{x} = \mathbf{x}^T \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T \mathbf{x}$$

Eigendecomposition (valid when $\mathbf{Q}$ has linearly independent eigenvectors).

Symmetric Matrices and Cost

- Cost term $\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k$ penalizes coordinates from zero, not necessarily axis-aligned.
- $\mathbf{Q}$ and $\mathbf{R}$ can be any positive-definite symmetric matrices, but diagonal matrices simpler in practice.

Non-Symmetric Matrices

- Consider $\mathbf{x}^T \mathbf{F} \mathbf{x}$ where $\mathbf{F}$ non-symmetric
- Decompose into symmetric and non-symmetric parts:

$$\mathbf{F} = \frac{\mathbf{F} + \mathbf{F}^T}{2} + \frac{\mathbf{F} - \mathbf{F}^T}{2}$$

$$\mathbf{x}^T \mathbf{F} \mathbf{x} = \mathbf{x}^T \frac{\mathbf{F} + \mathbf{F}^T}{2} \mathbf{x}$$

Only symmetric part matters; no point using non-symmetric $\mathbf{Q}$ and $\mathbf{R}$!

LQR Derivation

Solution has form:

$$\mathbf{U}^* = -\mathbf{H}^{-1} \mathbf{F}^T \mathbf{X} \mathbf{x}_0$$

where

$$\mathbf{H} = \begin{bmatrix} \mathbf{R} & \bar{\mathbf{S}} \\ \bar{\mathbf{S}}^T & \mathbf{X} \end{bmatrix}$$

and

$$\mathbf{F} = \begin{bmatrix} 2\mathbf{T}^T & \mathbf{X} \end{bmatrix} \begin{bmatrix} \mathbf{Q} & \mathbf{0} \\ \mathbf{0} & \mathbf{X} \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ \mathbf{T} \end{bmatrix}$$

Problem: Matrix sizes proportional to horizon length!

1. **Total cost:**

$$J(\mathbf{U}) = \mathbf{x}_0^T \begin{bmatrix} \mathbf{Q} & \cdots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ \mathbf{0} & \cdots & \mathbf{Q}_f \end{bmatrix} \begin{bmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} + \mathbf{U}^T \begin{bmatrix} \mathbf{R} & \cdots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ \mathbf{0} & \cdots & \mathbf{R} \end{bmatrix} \mathbf{U}$$

2. **System trajectory:**

$$\begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} = \begin{bmatrix} \mathbf{B} & \cdots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ \mathbf{A}^{N-1}\mathbf{B} & \cdots & \mathbf{B} \end{bmatrix} \mathbf{U} + \begin{bmatrix} \mathbf{A} \\ \vdots \\ \mathbf{A}^N \end{bmatrix} \mathbf{x}_0$$

Bellman Principle

Given optimal control sequence $\mathbf{U}^* = [\mathbf{u}_0^*, \dots, \mathbf{u}_{N-1}^*]$ and optimal trajectory $\mathbf{x}^*(k)$, sub-sequence $[\mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*]$ optimal for horizon $[t_k, N]$ starting from $\mathbf{x}^*(t_k)$.

Use this principle for iterative solution.

LQR Iterative Solution

Define **cost-to-go** as residual cost in $[t, N]$ starting from $\mathbf{x}_t$.

$$V_t(\mathbf{x}_t) = \min_{\mathbf{u}_t, \dots, \mathbf{u}_{N-1}} \sum_{k=t}^{N-1} [\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k] + \mathbf{x}_N^T \mathbf{Q}_f \mathbf{x}_N$$

Idea: Optimize last interval, roll-back in time.

LQR Iterative Solution (Continued)

Cost-to-go for last time-step:

$$V_N(\mathbf{x}_N) = \mathbf{x}_N^T \mathbf{Q}_f \mathbf{x}_N$$

Call this matrix $\mathbf{P}_N$.

For $N - 1$:

$$\begin{aligned} V_{N-1}(\mathbf{x}_{N-1}) &= \min_{\mathbf{u}_{N-1}} [\mathbf{x}_{N-1}^T \mathbf{Q} \mathbf{x}_{N-1} + \mathbf{u}_{N-1}^T \mathbf{R} \mathbf{u}_{N-1} + V_N(\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1})] \\ &= \min_{\mathbf{u}_{N-1}} [\mathbf{x}_{N-1}^T \mathbf{Q} \mathbf{x}_{N-1} + \mathbf{u}_{N-1}^T \mathbf{R} \mathbf{u}_{N-1} + (\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1})^T \mathbf{Q}_f (\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1})] \end{aligned}$$

Optimal $\mathbf{u}_{N-1}^*$ found by setting gradient w.r.t. $\mathbf{u}_{N-1}$ to zero.

$$\begin{aligned} \nabla_{\mathbf{u}_{N-1}} g_{N-1}(\mathbf{x}_{N-1}, \mathbf{u}_{N-1}) &= 2\mathbf{R} \mathbf{u}_{N-1} + 2\mathbf{B}^T \mathbf{Q}_f (\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1}) = \mathbf{0} \\ \mathbf{u}_{N-1}^* &= -(\mathbf{R} + \mathbf{B}^T \mathbf{Q}_f \mathbf{B})^{-1} \mathbf{B}^T \mathbf{Q}_f \mathbf{A} \mathbf{x}_{N-1} \mathbf{K}_{N-1} \mathbf{x}_{N-1} \end{aligned}$$

Closed-form solution found!

LQR Iterative Solution (Continued)

With optimal input for $N - 1$, compute exact cost-to V_{N-1} .

$$\begin{aligned} V_{N-1}(\mathbf{x}_{N-1}) &= \min_{\mathbf{u}_{N-1}} [\mathbf{x}_{N-1}^T \mathbf{Q} \mathbf{x}_{N-1} + \mathbf{u}_{N-1}^T \mathbf{R} \mathbf{u}_{N-1} + (\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1})^T \mathbf{Q}_f (\mathbf{A} \mathbf{x}_{N-1} + \mathbf{B} \mathbf{u}_{N-1})] \\ &= \mathbf{x}_{N-1}^T \left[\mathbf{Q} - \mathbf{A}^T \mathbf{Q}_f \mathbf{B} (\mathbf{R} + \mathbf{B}^T \mathbf{Q}_f \mathbf{B})^{-1} \mathbf{B}^T \mathbf{Q}_f \mathbf{A} + \mathbf{A}^T \mathbf{Q}_f \mathbf{A} \right] \mathbf{x}_{N-1} \\ &= \mathbf{x}_{N-1}^T \mathbf{P}_{N-1} \mathbf{x}_{N-1} \end{aligned}$$

Cost-to-go has quadratic form; iterate back in time.

LQR: Iteration

1. Set $\mathbf{P}_N = \mathbf{Q}_f$
2. For $t = N, \dots, 1$ set

$$\mathbf{P}_{t-1} := \mathbf{Q} + \mathbf{A}^T \mathbf{P}_t \mathbf{A} - \mathbf{A}^T \mathbf{P}_t \mathbf{B} (\mathbf{R} + \mathbf{B}^T \mathbf{P}_t \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P}_t \mathbf{A}$$

3. For $t = 0, \dots, N - 1$ set

$$\mathbf{K}_t := -(\mathbf{R} + \mathbf{B}^T \mathbf{P}_{t+1} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P}_{t+1} \mathbf{A}$$

4. For $t = 0, \dots, N - 1$ optimal input:

$$\mathbf{u}_t^* = \mathbf{K}_t \mathbf{x}_t$$

- Cost-to-go matrix: $V_t(\mathbf{x}_t) = \mathbf{x}_t^T \mathbf{P}_t \mathbf{x}_t$
- Optimal control input matrix

LQR: Infinite Horizon

- Infinite horizon LQR control:

$$\mathbf{u} = \mathbf{K} \mathbf{x}_0$$

where

$$\mathbf{K} = -(\mathbf{R} + \mathbf{B}^T \mathbf{P} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P} \mathbf{A}$$

$$\mathbf{P} = \mathbf{Q} + \mathbf{A}^T \mathbf{P} \mathbf{A} - \mathbf{A}^T \mathbf{P} \mathbf{B} (\mathbf{R} + \mathbf{B}^T \mathbf{P} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P} \mathbf{A}$$

Discrete-time **algebraic Riccati equation** (ARE).

- $\mathbf{K}$ and $\mathbf{P}$ time-invariant.
- Solve ARE directly (no recursion).

Example: LQR for Cart-Pole

Cart-Pole Configuration

Configuration: $\mathbf{q} = [x, \theta]^T$

Goal: Regulate to $\mathbf{q}_d = [0, \pi/2]^T$ with $\dot{\mathbf{q}}_d = [0, 0]^T$.

Energy:

$$T = \frac{1}{2} m_c \dot{x}^2 + \frac{1}{2} m_p (\dot{x} + l \dot{\theta} \cos \theta)^2 + \frac{1}{2} m_p (l \dot{\theta})^2$$

$$U = -m_p g l \cos \theta$$

Equations of motion (Lagrangian):

$$(m_c + m_p)\ddot{x} + m_pl\ddot{\theta} \cos \theta - m_pl\dot{\theta}^2 \sin \theta = f$$

$$m_pl\ddot{x} \cos \theta + m_pl^2\ddot{\theta} + m_pgl \sin \theta = 0$$

$$\mathbf{B}_q = \begin{bmatrix} m_c + m_p & m_pl \cos \theta \\ m_pl \cos \theta & m_pl^2 \end{bmatrix}$$

$$\mathbf{C}_q(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} 0 & -m_pl\dot{\theta} \sin \theta \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{g}(\mathbf{q}) = \begin{bmatrix} 0 \\ m_pgl \sin \theta \end{bmatrix}$$

$$\mathbf{u} = \begin{bmatrix} f \\ 0 \end{bmatrix}$$

Example: LQR for Cart-Pole (Continued)

Apply LQR by linearizing model around desired equilibrium using Taylor expansion.

Extended state $\mathbf{x}_e = [\mathbf{q}, \dot{\mathbf{q}}]^T$:

$$\dot{\mathbf{x}}_e = \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{B}_q^{-1}(\mathbf{u} - \mathbf{C}_q(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q})) \end{bmatrix} = \mathbf{h}(\mathbf{x}_e, \mathbf{u})$$

Equilibrium $(\mathbf{x}_e^*, \mathbf{u}^*)$: $\mathbf{h}(\mathbf{x}_e^*, \mathbf{u}^*) = \mathbf{0}$.

$$\Delta \dot{\mathbf{x}}_e \approx \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}_e} \right|_{(\mathbf{x}_e^*, \mathbf{u}^*)} (\mathbf{x}_e - \mathbf{x}_e^*) + \left. \frac{\partial \mathbf{h}}{\partial \mathbf{u}} \right|_{(\mathbf{x}_e^*, \mathbf{u}^*)} (\mathbf{u} - \mathbf{u}^*) = \mathbf{A}(\mathbf{x}_e - \mathbf{x}_e^*) + \mathbf{B}(\mathbf{u} - \mathbf{u}^*)$$

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{B}_q^{-1} \frac{\partial \mathbf{g}}{\partial \mathbf{q}} & -\mathbf{B}_q^{-1} \mathbf{C}_q(\mathbf{x}_e^*, \mathbf{u}^*) \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} \mathbf{0} \\ \mathbf{B}_q^{-1} \end{bmatrix}$$

Example: LQR for Cart-Pole (Continued)

- Try changing input penalty (fuel cost).

Lecture 3b: Trajectory Tracking

Trajectory Tracking

Often important not only to reach goal, but how to reach it.

Desired trajectory $\mathbf{q}_d(t)$ (or $\mathbf{x}_d(t)$ in task space) typically differentiable and feasible (compliant with system requirements).

PD (PID) Control

Similarly for regulation, use PD or PID to stabilize around trajectory.

- **Feedback term:**

$$\mathbf{u}_{fb} = \mathbf{K}_p(\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}})$$

- No model knowledge required.
- Purely reactive (error-driven), no error anticipation.
- Add **integral term** to compensate steady-state error.

Dynamic Inversion

- Robot with canonical dynamics:

$$\mathbf{B}_q \ddot{\mathbf{q}} + \mathbf{n}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{u}$$

- Twice differentiable desired trajectory $t \in [0, T]: \mathbf{q}_d(t) \rightarrow \dot{\mathbf{q}}_d(t), \ddot{\mathbf{q}}_d(t)$
- By **inverting dynamics**, compute input that tracks trajectory in nominal conditions starting from desired state.

Dynamic Inversion (Continued)

$$\mathbf{B}_q \ddot{\mathbf{q}} + \mathbf{n}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{u}$$

model

Dynamic inversion trajectory $\mathbf{q}_d(t), \dot{\mathbf{q}}_d(t), \ddot{\mathbf{q}}_d(t)$

Feedforward input:

$$\mathbf{u}_{ff} = \mathbf{B}_q \ddot{\mathbf{q}}_d + \mathbf{n}(\mathbf{q}_d, \dot{\mathbf{q}}_d)$$

- **Feedforward input** allows robot to follow trajectory if starts exactly on initial state ($\mathbf{q}_0 = \mathbf{q}_{d0}, \dot{\mathbf{q}}_0 = \dot{\mathbf{q}}_{d0}, \ddot{\mathbf{q}}_0 = \ddot{\mathbf{q}}_d(0)$), model known perfectly, no disturbances.
- In practice unrealistic; need **feedback information** (PD or PID).

Feedforward + PD (PID)

- **Feedforward + PD control:** $\mathbf{u} = \mathbf{u}_{ff} + \mathbf{u}_{fb}$.
- Combine imperfect feedforward with PD control.

```
# Example: Feedforward + PD control implementation
def feedforward_pd_control(q_d, q_dot_d, q_ddot_d, q, q_dot, B_q, n_func, K_p, K_d):
    """
    Compute feedforward + PD control input

    Args:
        q_d: desired joint positions
        q_dot_d: desired joint velocities
```

```

q_ddot_d: desired joint accelerations
q: current joint positions
q_dot: current joint velocities
B_q: inertia matrix
n_func: function to compute n(q, q_dot)
K_p: proportional gain matrix
K_d: derivative gain matrix

```

Returns:

```

    u: control input torques
"""
# Feedforward term
u_ff = B_q @ q_ddot_d + n_func(q_d, q_dot_d)

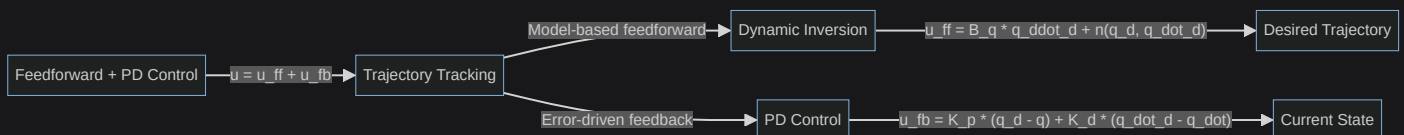
# PD feedback term
u_fb = K_p @ (q_d - q) + K_d @ (q_dot_d - q_dot)

# Total control input
u = u_ff + u_fb

return u

```

This approach combines the model-based feedforward term with error-driven feedback for robust trajectory tracking.



References

- Robot Dynamics and Control
- Linear Systems Theory
- Optimal Control Theory
- Lyapunov Stability Analysis